



Univerzitet u Nišu
Elektronski fakultet



SEMINARSKI RAD
IZ PREDMETA METODI I SISTEMI ZA OBRADU SIGNALA
Tema: ***Analiza i implementacija rečničkih metoda kompresije***

Mentor:

Prof. Miloš Radmanović

Studenti:

Mićović Milan 19260

Mitić Aleksa 19252

Niš, 2026. godine

SADRŽAJ

1. Uvod	3
2. Podela algoritama kompresije	4
3. Tehnike kompresije zasnovane na rečniku.....	5
4. LZ familija algoritama.....	7
5. LZ78 algoritam.....	8
6. LZW algoritam	9
7. Format kompresovanog zapisa	11
8. Implementacija projekta	12
9. Metode za kompresiju podataka	13
9.1 LZ78 kompresija u Python implementaciji	13
9.2 LZW kompresija u Python implementaciji.....	14
9.3 LZW kompresija u Java implementaciji.....	15
10. Metode za dekompresiju podataka	17
10.1 LZ78 dekompresija	17
10.2 LZW dekompresija	18
11. Rezultati izvršenja programa.....	19
12. Poređenje LZ78 i LZW algoritma	20
13. Moguća unapređenja	21
14. Zaključak.....	22
15. Literatura	23

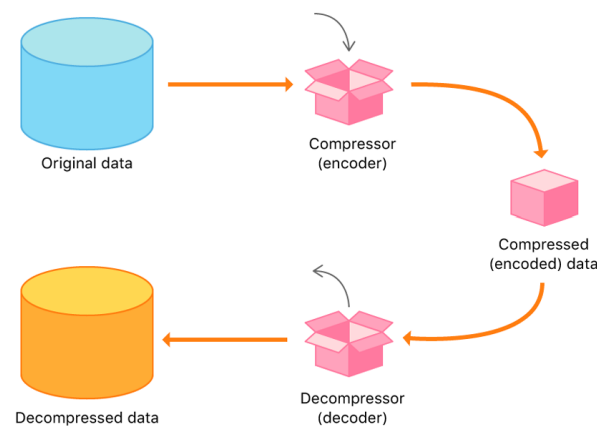
1. Uvod

Kompresija podataka predstavlja postupak kojim se ulazni tok podataka pretvara u drugi tok podataka manje veličine. Cilj kompresije je smanjenje prostora potrebnog za skladištenje ili smanjenje vremena potrebnog za prenos. U računarskim sistemima, telekomunikacijama i sistemima za obradu signala kompresija ima značajnu ulogu zato što se realni podaci često čuvaju i prenose u velikim količinama.

Najjednostavnije rečeno, kompresija podataka je postupak sažimanja jedne ili više datoteka u novi zapis koji bi trebalo da bude manji od početnog zapisa. Ovaj postupak ne zavisi samo od izabranog algoritma, već i od prirode samih podataka. Tekstualni fajlovi, programski kod i log fajlovi često sadrže mnogo ponavljanja. Kod takvih podataka moguće je dobiti dobar stepen kompresije. Sa druge strane, već kompresovani fajlovi, kriptovani podaci ili podaci koji nemaju ponavljanja uglavnom se teško dodatno kompresuju.

Kod kompresije bez gubitaka dekomprimovani fajl mora biti identičan originalu. To znači da se posle dekompresije dobija isti niz bajtova koji je postojao pre kompresije. Ovaj zahtev je neophodan kod tekstualnih dokumenata, izvršnih fajlova, arhiva, baza podataka i konfiguracionih fajlova. Tema ovog projekta pripada upravo kompresiji bez gubitaka.

U radu su obrađene rečničke metode kompresije podataka, sa posebnim osvrtom na LZ78 i LZW algoritam. Obe metode pripadaju Lempel-Ziv porodici algoritama. Osnovna ideja je da se ponavljajući nizovi simbola ne zapisuju uvek u izvornom obliku, već se zamenjuju kraćim referencama na rečnik. Rečnik se gradi tokom rada algoritma, pa kompresor i dekompresor ne moraju unapred da razmenjuju kompletan rečnik.



Slika 1: Kompresija i dekompresija podataka

2. Podela algoritama kompresije

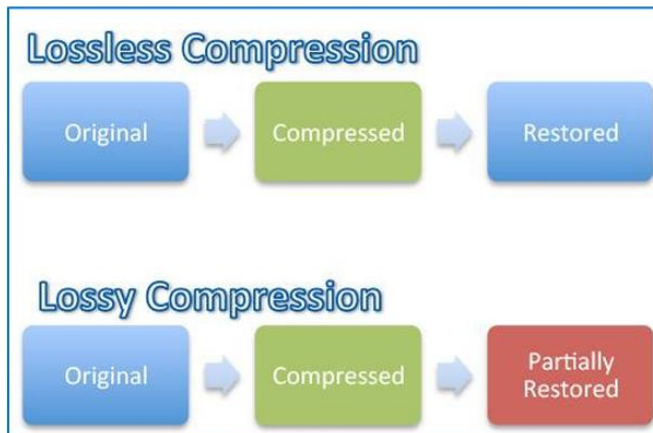
Postoji veliki broj algoritama za kompresiju podataka. Ipak, prema tome da li se prilikom dekompresije dobija potpuno isti sadržaj ili samo približna rekonstrukcija, algoritmi se najčešće dele u dve grupe:

- algoritmi kompresije bez gubitaka
- algoritmi kompresije sa gubicima

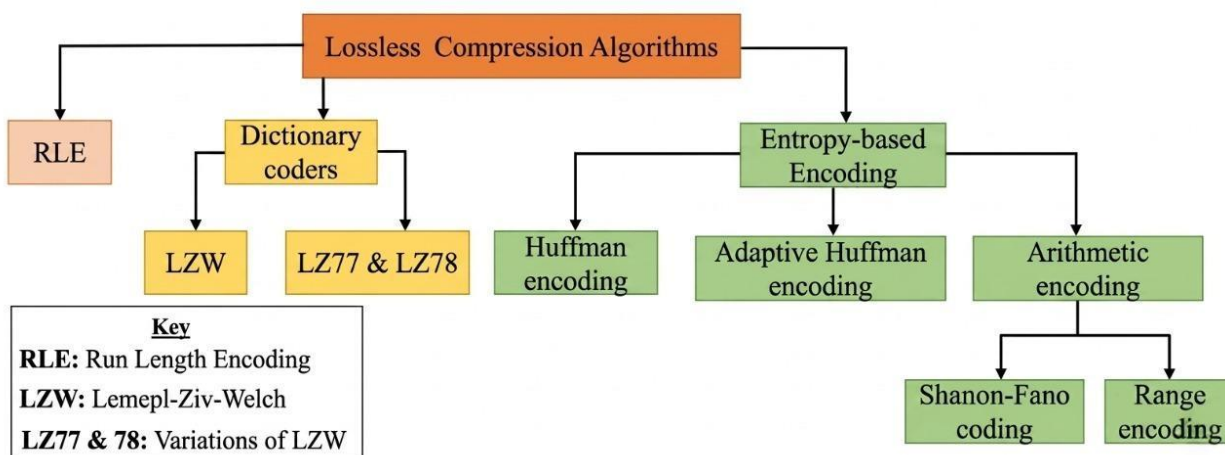
Kompresija bez gubitaka predstavlja sažimanje podataka pri kome se ne gubi nijedan deo informacije. Originalni i dekomprimovani fajl moraju biti isti. Ova grupa algoritama koristi redundansu u podacima. Redundansa je višak informacija koji se može predstaviti kraćim zapisom. Primeri kompresije bez gubitaka su ZIP, PNG, GIF, Huffmanovo kodiranje, aritmetičko kodiranje i Lempel-Ziv algoritmi.

Kompresija sa gubicima dozvoljava određeno odstupanje od originalnog signala. Takvi algoritmi se koriste kod multimedijalnih podataka, na primer kod slike, zvuka i videa. Razlog je u tome što ljudsko oko ili uho ne primećuju svaku promenu u signalu, pa se odbacivanjem manje važnih informacija može postići mnogo veći stepen kompresije. Primeri su JPEG, MP3, AAC i različiti video kodeci.

Za ovaj projekat važna je kompresija bez gubitaka. LZ78 i LZW nisu namenjeni tome da aproksimiraju signal, već da pronađu ponavljanja i da ih zapišu kraćim kodovima.



Slika 2: Podela algoritama kompresije



Slika 3: Lossless algoritmi kompresije

3. Tehnike kompresije zasnovane na rečniku

Algoritmi kompresije zasnovani na rečniku ne posmatraju svaki simbol potpuno odvojeno. Umesto toga, oni traže duže nizove simbola koji se ponavljaju. Kada se isti niz ponovo pojavi, algoritam ga može zameniti indeksom, kodom ili pokazivačem na mesto u rečniku.

Rečnik u ovim metodama može biti statički ili dinamički. Statički rečnik je unapred poznat i ne menja se tokom rada. Dinamički rečnik se gradi tokom prolaska kroz ulazne podatke. LZ78 i LZW koriste dinamički rečnik, što ih čini adaptivnim metodama. Algoritam se prilagođava sadržaju fajla koji se trenutno obrađuje.

Kod rečničke kompresije kompresor i dekompresor moraju graditi rečnik na isti način. Ako oba dela sistema koriste ista pravila, nije potrebno čuvati ceo rečnik u kompresovanom fajlu. Dovoljno je čuvati tok kodova ili tokena. Dekompresor zatim iz tog toka obnavlja originalni sadržaj.

Kompresioni odnos se najčešće računa izrazom:

Kompresioni odnos

$$= \frac{\text{Velicina kompresovanog fajla}}{\text{Velicina originalnog fajla}}$$

Ako je odnos manji od 1, kompresovani fajl je manji od originalnog. Ako je odnos veći od 1, algoritam nije uspeo da smanji veličinu fajla. To ne znači da je algoritam neispravan. Kod malih fajlova dodatni trošak zaglavlja, kodova i tokena može biti veći od dobitka. Takođe, ako ulazni podaci nemaju ponavljanja, rečnik ne može da pronađe korisne obrasce.

Primer ideje rečničke kompresije može se prikazati na tekstu:

ASK NOT WHAT YOUR COUNTRY CAN DO FOR YOU

ASK WHAT YOU CAN DO FOR YOUR COUNTRY

Umesto da se svaka rec iznova zapisuje u punom obliku, algoritam pri prvom pojavljivanju dodaje rec u rečnik i dodeljuje joj jedinstveni indeks(broj). Svako naredno pojavljivanje iste reci u tekstu zamenjuje se tim brojem. Kao što se vidi na slici, dugacke reci poput "COUNTRY" zamenjene su jednocifrenim brojem, čime se značajno smanjuje količina podataka potrebnih za zapis teksta.

The screenshot displays a text compression interface. On the left, a dictionary titled "REČNIK:" lists words with their corresponding indices: 1. ASK, 2. NOT, 3. WHAT, 4. YOUR, 5. COUNTRY, 6. CAN, 7. DO, 8. FOR, and 9. YOU. On the right, the "ORIGINALNI TEKST:" is shown as "ASK NOT WHAT YOUR COUNTRY CAN DO FOR YOU ASK WHAT YOU CAN DO FOR YOUR COUNTRY". Below this, the "ENKODOVAN TEKST:" is shown as a sequence of numbers: "1 2 3 4 5 6 7 8 9 1 3" on the first line and "4 5" on the second line, representing the encoded version of the original text.

Slika 4: Primer enkodovanja koriscenjem recnika

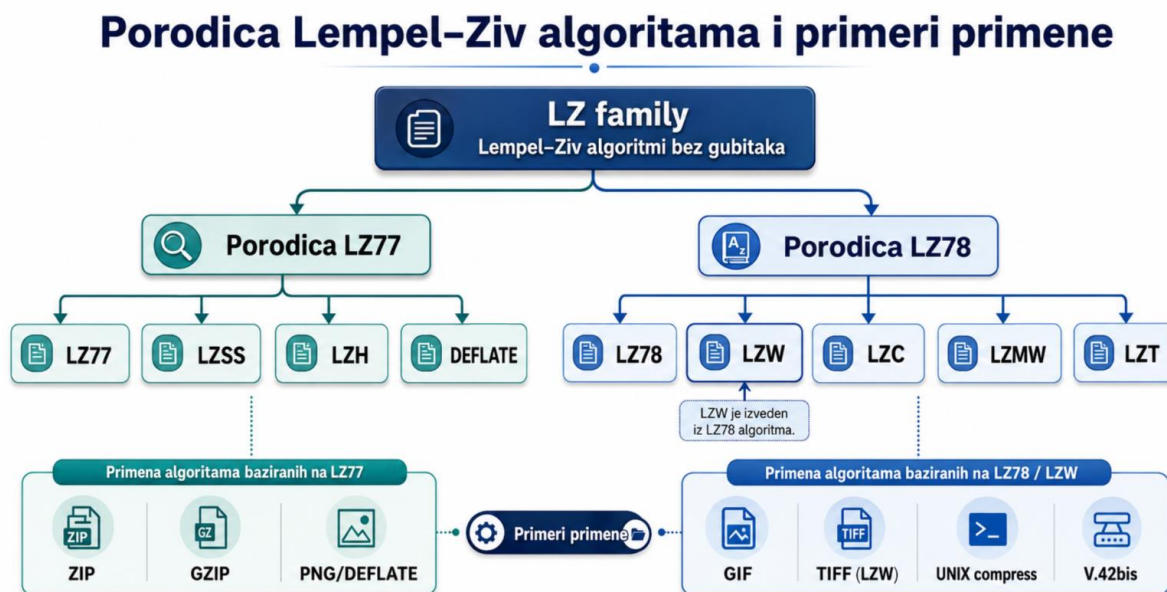
4. LZ familija algoritama

Abraham Lempel i Jacob Ziv su postavili osnovu velike porodice algoritama za kompresiju bez gubitaka. Njihovi radovi iz 1977. i 1978. godine doveli su do algoritama LZ77 i LZ78. Na osnovu tih algoritama razvijene su mnoge praktične varijante koje se i danas koriste u arhiviranju, grafičkim formatima i komunikacionim sistemima.

LZ algoritmi vrše kompresiju tako što ponovljene sekvence zamenjuju referencama na ranije viđene sekvence. Razlikuju se po tome kako pamte prethodno viđene podatke i kako zapisuju referencu.

LZ77 koristi klizni prozor. Kompresor traži da li se trenutni niz već nalazi u prozoru prethodno pročitanih znakova. Ako postoji poklapanje, zapisuje se par ili trojka koja opisuje udaljenost i dužinu poklapanja. LZ78 koristi drugačiji pristup. Umesto prozora, on gradi rečnik fraza. Svaka nova fraza se dodaje u rečnik i kasnije može biti referencirana indeksom.

LZW je praktično unapređenje LZ78 algoritma. Kod LZ78 izlazni token sadrži indeks prefiksa i naredni simbol. Kod LZW algoritma početni rečnik već sadrži sve pojedinačne simbole, pa kompresor može da šalje samo kodove iz rečnika. Zbog toga LZW često daje kompaktniji izlaz.



Slika 5: Porodica LZ algoritama i primeri primene

5. LZ78 algoritam

LZ78 algoritam koristi rečnik koji se gradi tokom kompresije. Rečnik sadrži fraze koje su već viđene u ulaznom toku. Kada kompresor pronade najdužu frazu koja postoji u rečniku, on zapisuje indeks te fraze i prvi sledeći simbol koji tu frazu proširuje.

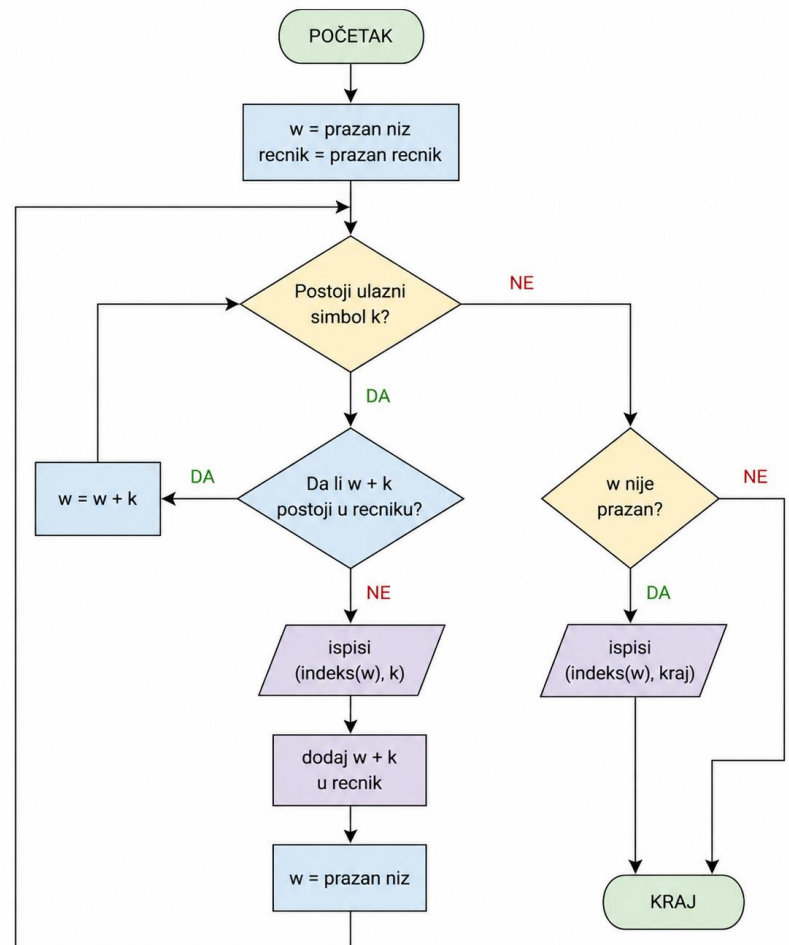
Izlaz LZ78 algoritma je niz tokena:

(indeks_prefiksa, sledeci_simbol)

Indeks prefiksa pokazuje na unos u rečniku. Vrednost 0 označava prazan prefiks. Sledeći simbol je bajt koji proširuje prefiks i formira novu frazu. Posle upisa tokena nova fraza se dodaje u rečnik.

Pseudokod LZ78 kompresije:

```
w = prazan niz
recnik = prazan rečnik
dok postoji ulazni simbol k:
    ako w + k postoji u rečniku:
        w = w + k
    inace:
        ispisi (indeks(w), k)
        dodaj w + k u rečnik
        w = prazan niz
    ako w nije prazan:
        ispisi (indeks(w), kraj)
```



Dekompresija radi obrnutim redosledom, ali koristi ista pravila za građenje rečnika. Kada dekompresor pročita token, on uzima frazu iz rečnika preko indeksa prefiksa i dodaje sledeći simbol. Dobijena fraza se dodaje na izlaz i upisuje kao novi unos u rečnik.

Prednost LZ78 algoritma je jednostavna konstrukcija rečnika. Algoritam jasno pokazuje kako se ponavljanja pretvaraju u fraze. Mana je to što svaki token čuva dva podatka. Ako je fajl mali ili ako ima malo ponavljanja, izlaz može biti veći od originala.

LZ78 algoritam kompresije



Slika 6: Princip rada LZ78 algoritma

6. LZW algoritam

LZW algoritam je razvio Terry Welch kao praktično unapređenje LZ78 algoritma. Algoritam je adaptivan i koristi rečnik koji se širi tokom kompresije. Za razliku od LZ78, početni rečnik nije prazan. U bajt-orijentisanoj implementaciji početni rečnik sadrži svih 256 mogućih vrednosti bajta.

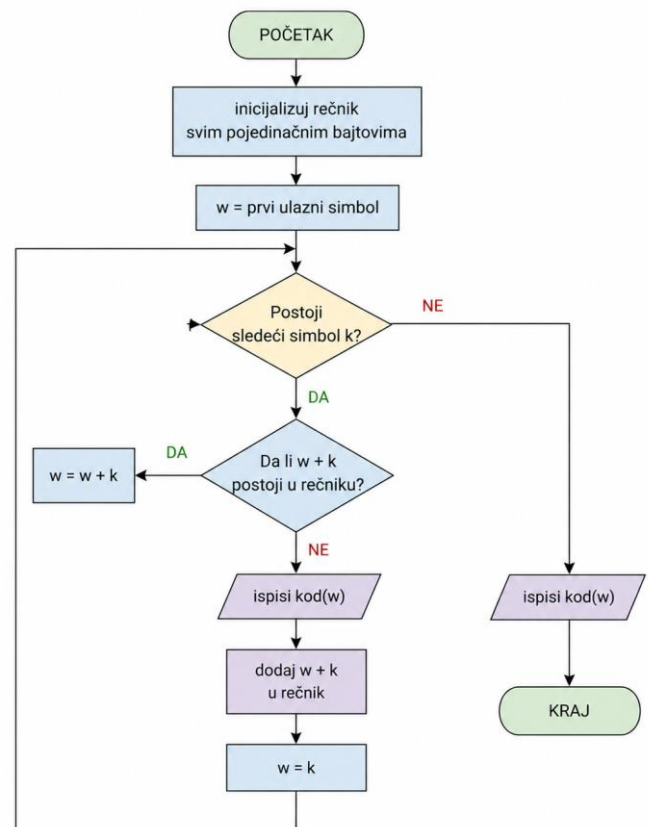
Kompresor uvek traži najduži niz koji se već nalazi u rečniku. Kada pronade takav niz, ispisuje njegov kod. Zatim u rečnik dodaje taj niz proširen narednim simbolom. Zbog toga se izlaz sastoji samo od kodova, a ne od parova indeks-simbol.

Pseudokod LZW kompresije:

```

inicijalizuj rečnik bajtovima
w = prvi ulazni simbol
dok postoji sledeći simbol k:
    ako w + k postoji u rečniku:
        w = w + k
    inace:
        ispisi kod(w)
        dodaj w + k u rečnik
        w = k
    ispisi kod(w)
  
```

LZW dekompresor počinje od istog početnog rečnika. Prvi kod se direktno prevodi u simbol ili frazu. Za svaki sledeći kod dekompresor pronalazi odgovarajući unos u rečniku i dodaje ga na izlaz.



Zatim u rečnik dodaje prethodnu frazu proširenu prvim simbolom trenutne fraze.

Poseban slučaj se javlja kada kompresor pošalje kod koji dekompresor još nije stigao da upiše u rečnik. Taj slučaj se rešava pravilom:

`unos = prethodna_fraza + prvi_simbol(prethodna_fraza)`

Ovaj slučaj nije greška, već normalna posledica načina na koji se rečnik gradi kod LZW algoritma.

LZW je korišćen u više praktičnih sistema, uključujući GIF format i starije arhivere. U našem projektu koristi se fiksna širina koda od 12 bitova, što odgovara demonstracionoj varijanti sa najviše 4096 unosa u rečniku.

LZW algoritam kompresije



Slika 7: Princip rada LZW algoritma kompresije

7. Format kompresovanog zapisa

U projektu je implementiran jednostavan projektni format kompresovanog fajla. Cilj formata nije kompatibilnost sa industrijskim arhiverima, već jasna demonstracija algoritama i mogućnost merenja rezultata.

Python implementacija koristi zaglavlje koje sadrži:

- oznaku formata
- oznaku algoritma
- originalnu veličinu fajla
- broj tokena ili kodova

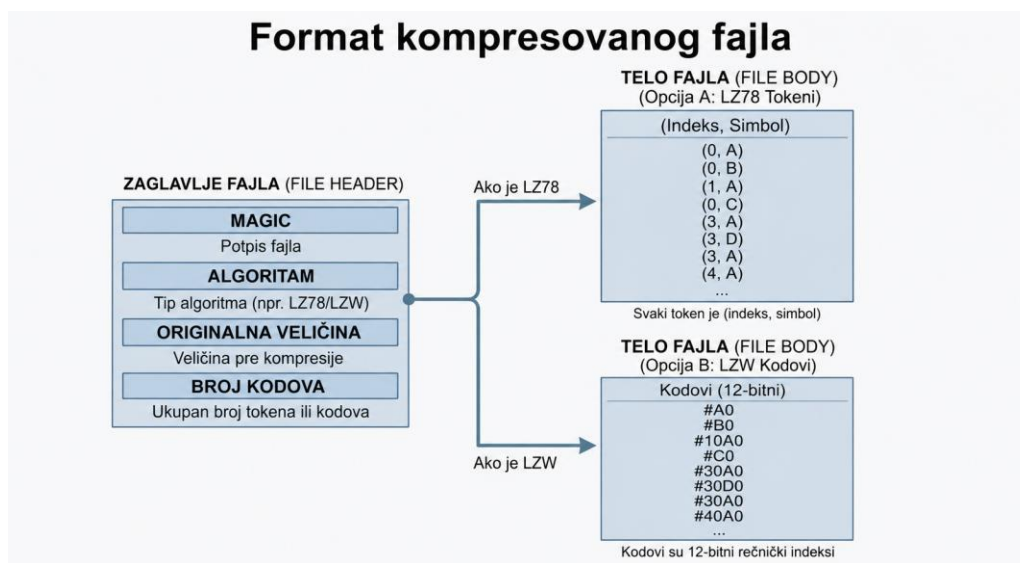
Oznaka formata je RKC26. Oznaka algoritma ima vrednost 1 za LZ78 i vrednost 2 za LZW. Originalna veličina je potrebna da bi dekompresor mogao da proveri da li je rekonstruisani izlaz ispravne dužine. Broj tokena ili kodova omogućava lakše čitanje tela fajla.

Kod LZ78 algoritma svaki token se čuva kao dva 16-bitna podatka:

indeks_prefiksa, sledeci_bajt

Vrednost 256 koristi se kao marker za poslednji token kada postoji prefiks bez sledećeg bajta. Ovakav zapis je jednostavan za razumevanje, ali nije maksimalno efikasan.

Kod LZW algoritma kodovi se pakuju kao 12-bitne vrednosti. To znači da svaki kod može imati vrednost od 0 do 4095. Početnih 256 kodova predstavlja pojedinačne bajtove, a preostali kodovi se dodeljuju frazama koje algoritam pronalazi tokom kompresije.

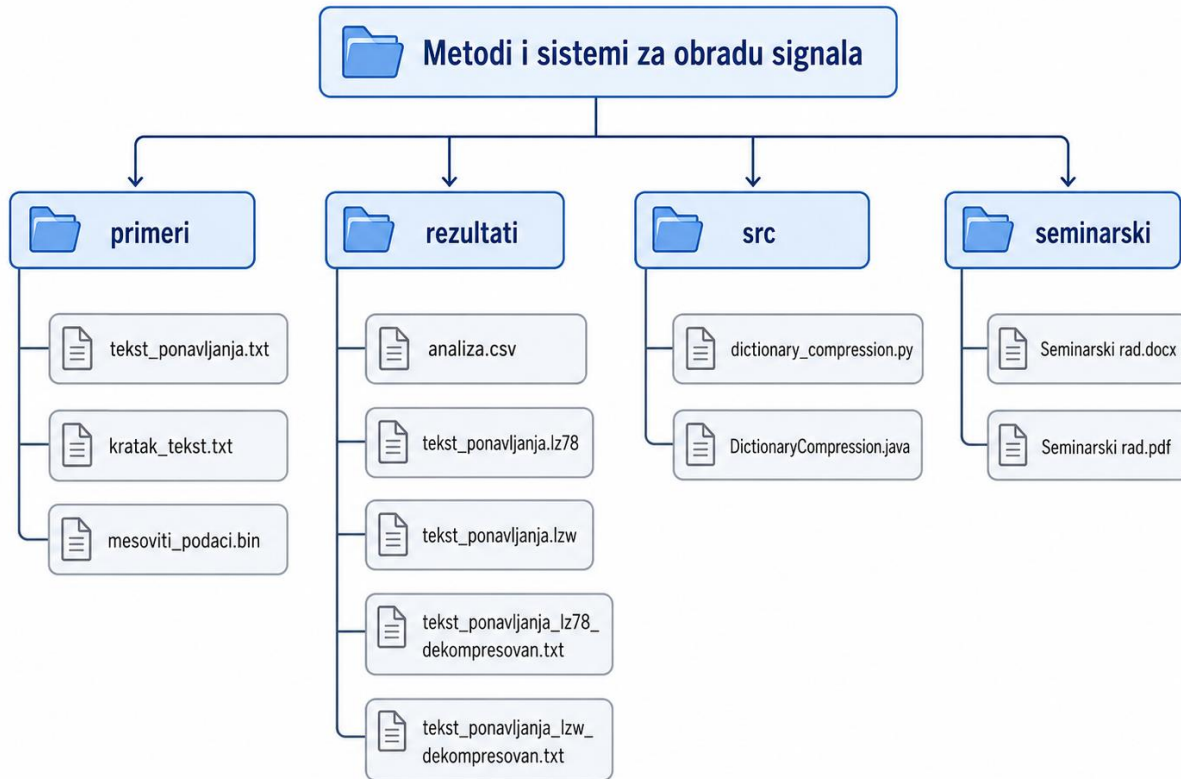


Slika 8: Format kompresovanog fajla

8. Implementacija projekta

Postoji teorijski deo, Python implementacija, Java implementacija, primeri za testiranje i rezultati.

Struktura seminarskog projekta



Slika 9:Struktura seminarskog projekta

Python implementacija se nalazi u fajlu src/dictionary_compression.py. Ona radi nad bajtovima, pa može da obradi tekstualne i binarne fajlove. Program podržava tri režima rada: kompresiju, dekompresiju i analizu.

Java implementacija se nalazi u fajlu src/DictionaryCompression.java. Ona je napisana kao demonstracioni program, slično kodu iz starog projekta. Java deo pravi čitljive fajlove sa kodovima i poseban fajl sa dekomprimovanim sadržajem.

U obe implementacije koristi se isti osnovni princip: rečnik se gradi tokom prolaska kroz ulazne podatke, a dekompresija je moguća zato što dekompresor ponavlja ista pravila za građenje rečnika.

9. Metode za kompresiju podataka

U ovom poglavlju biće detaljno predstavljene implementacije rečničkih metoda kompresije, sa posebnim fokusom na LZ78 i LZW algoritme. Kroz prikaz Python i Java koda, ilustrovaće se praktična primena teorijskih koncepata objašnjenih u prethodnim delovima rada. Ove implementacije demonstriraju kako se rečnici dinamički grade i koriste za efikasno predstavljanje ponavljajućih obrazaca u podacima, što je temelj za postizanje kompresije bez gubitaka.

9.1 LZ78 kompresija u Python implementaciji

U Python implementaciji LZ78 kompresija prima niz bajtova i vraća kompresovan zapis zajedno sa veličinom rečnika. Rečnik mapira fraze na njihove indekse. Trenutna fraza se proširuje sve dok postoji u rečniku. Kada proširena fraza nije pronađena, u izlaz se dodaje token i nova fraza se upisuje u rečnik.

```
def lz78_compress(data: bytes) -> tuple[bytes, int]:
    dictionary: dict[bytes, int] = {}
    next_index = 1
    current = b""
    tokens: list[tuple[int, int]] = []

    for byte in data:
        candidate = current + bytes([byte])
        if candidate in dictionary:
            current = candidate
            continue

        prefix_index = dictionary.get(current, 0)
        tokens.append((prefix_index, byte))
        dictionary[candidate] = next_index
        next_index += 1
        current = b""

    if current:
        tokens.append((dictionary[current], 256))
```

Ovaj deo koda pokazuje osnovnu ideju LZ78 algoritma. Ako kandidat već postoji u rečniku, algoritam nastavlja da proširuje trenutnu frazu. Ako kandidat ne postoji, zapisuje se indeks postojećeg prefiksa i bajt koji ga proširuje.

9.2 LZW kompresija u Python implementaciji

Kod LZW algoritma početni rečnik sadrži svih 256 bajtova. Promenljiva `current` čuva najduži trenutno poznati niz. Kada se pojavi kandidat koji nije u rečniku, ispisuje se kod za `current`, a kandidat se dodaje u rečnik.

```
def lzw_compress(data: bytes, max_bits: int = LZW_CODE_BITS) -> tuple[bytes, int]:
    max_dictionary_size = 1 << max_bits
    dictionary = {bytes([i]): i for i in range(256)}
    next_code = 256
    current = bytes([data[0]])
    codes: list[int] = []

    for byte in data[1:]:
        candidate = current + bytes([byte])
        if candidate in dictionary:
            current = candidate
            continue

        codes.append(dictionary[current])
        if next_code < max_dictionary_size:
            dictionary[candidate] = next_code
            next_code += 1
        current = bytes([byte])

    codes.append(dictionary[current])
```

Ova implementacija koristi najviše 4096 unosa u rečniku. To ograničenje odgovara 12-bitnim kodovima. Kada se rečnik popuni, novi unosi se više ne dodaju, ali se kompresija nastavlja pomoću postojećeg rečnika.

9.3 LZW kompresija u Java implementaciji

Java implementacija koristi HashMap za rečnik i ArrayList<Integer> za izlazne kodove.

```
public static List<Integer> lzwCompress(byte[] input) {
    Map<String, Integer> dictionary = new HashMap<>();
    List<Integer> result = new ArrayList<>();
    String text = new String(input, StandardCharsets.ISO_8859_1);

    for (int i = 0; i < 256; i++) {
        dictionary.put(String.valueOf((char) i), i);
    }

    String current = "";
    int nextCode = 256;

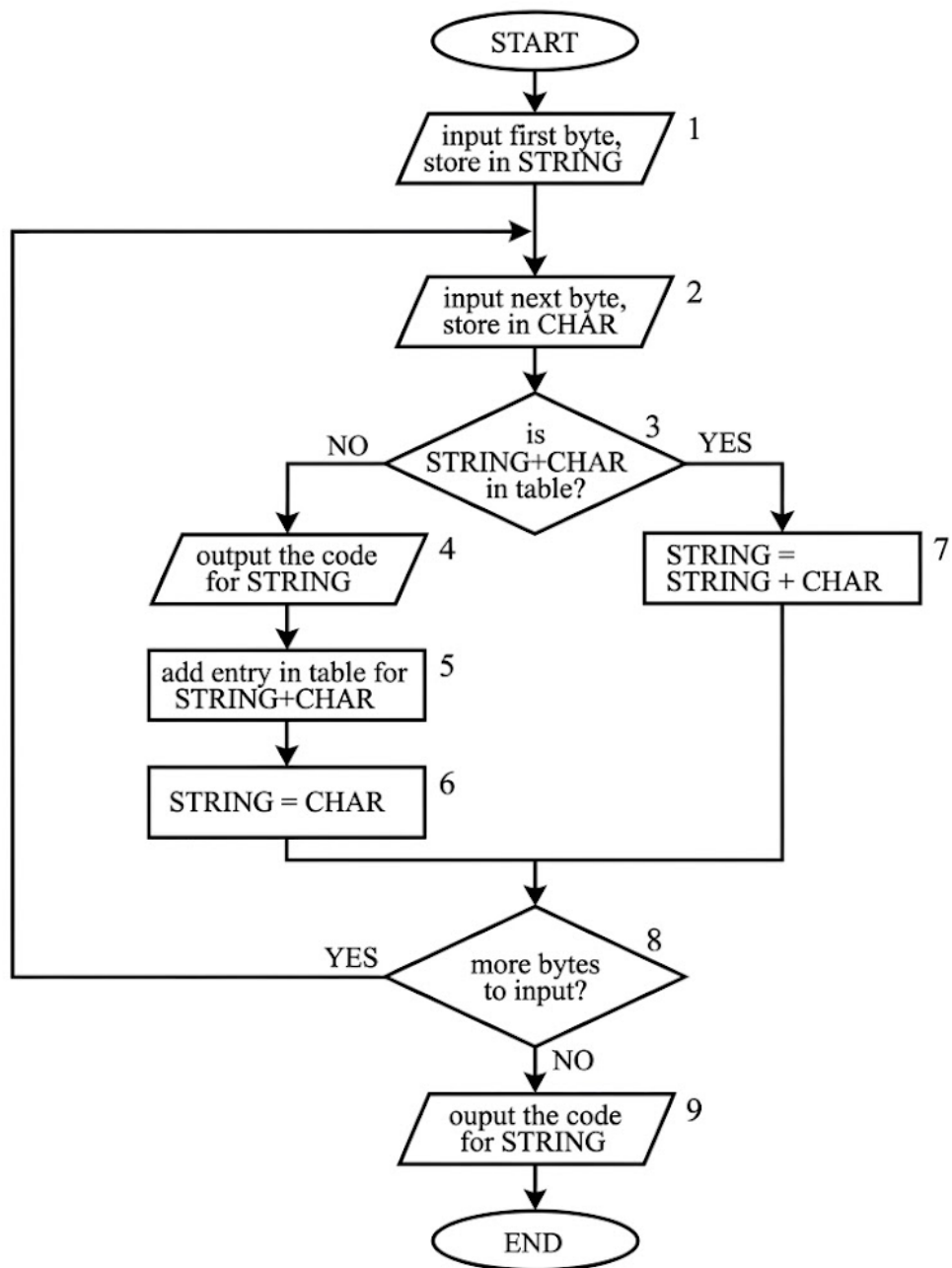
    for (int i = 0; i < text.length(); i++) {
        char character = text.charAt(i);
        String candidate = current + character;

        if (dictionary.containsKey(candidate)) {
            current = candidate;
        } else {
            result.add(dictionary.get(current));
            if (nextCode < LZW_MAX_DICTIONARY_SIZE) {
                dictionary.put(candidate, nextCode++);
            }
            current = String.valueOf(character);
        }
    }

    if (!current.isEmpty()) {
        result.add(dictionary.get(current));
    }

    return result;
}
```

Za razliku od Python programa koji pravi binarni projektni format, Java program čuva kodove u tekstualnom fajlu. To olakšava proveru rada algoritma i odgovara demonstracionom karakteru projekta.



Slika 10: Dijagram toka LZW kompresije

10. Metode za dekompresiju podataka

Nakon procesa kompresije, neophodno je omogućiti preciznu rekonstrukciju originalnih podataka. U ovom poglavlju biće prikazane implementacije dekompresionih algoritama za LZ78 i LZW metode, koje prate inverzne logike kompresije. Detaljno će se objasniti kako se na osnovu kompresovanih tokena ili kodova, uz pomoć dinamički izgrađenog rečnika, originalni podaci bez gubitaka vraćaju u prvobitni oblik.

10.1 LZ78 dekompresija

Kod LZ78 dekompresije rečnik mapira indekse na fraze. Za svaki token čita se indeks prefiksa i sledeći bajt. Fraza se dobija spajanjem prefiksa i sledećeg bajta. Zatim se fraza dodaje na izlaz i upisuje u rečnik.

```
def lz78_decompress(payload: bytes) -> bytes:
    algorithm_id, original_size, token_count, body = _unpack_header(payload)
    dictionary: dict[int, bytes] = {0: b""}
    next_index = 1
    result = bytearray()

    for offset in range(0, len(body), 4):
        prefix_index, next_byte = struct.unpack(">HH", body[offset : offset + 4])
        phrase = dictionary[prefix_index]
        if next_byte != 256:
            phrase += bytes([next_byte])

        result.extend(phrase)
        dictionary[next_index] = phrase
        next_index += 1
```

U implementaciji se proverava da li dužina dekomprimovanog izlaza odgovara originalnoj veličini iz zaglavlja. Time se otkriva greška u fajlu, pogrešan algoritam ili neispravan tok tokena.

10.2 LZW dekompresija

LZW dekompresija koristi početni rečnik sa 256 pojedinačnih bajtova. Prvi kod se direktno prevodi u frazu. Svaki sledeći kod daje novu frazu, a dekompresor zatim dodaje novi unos u rečnik.

```
def lzw_decompress(payload: bytes) -> bytes:
    algorithm_id, original_size, token_count, body = _unpack_header(payload)
    codes = _unpack_fixed_width_codes(body, LZW_CODE_BITS, token_count)

    dictionary = {i: bytes([i]) for i in range(256)}
    next_code = 256
    previous = dictionary[codes[0]]
    result = bytearray(previous)

    for code in codes[1:]:
        if code in dictionary:
            entry = dictionary[code]
        elif code == next_code:
            entry = previous + previous[:1]
        else:
            raise ValueError(f"Neispravan LZW kod: {code}")

        result.extend(entry)
        dictionary[next_code] = previous + entry[:1]
        next_code += 1
        previous = entry
```

Poseban slučaj `code == next_code` je važan za ispravnost algoritma. On se javlja kada kompresor pošalje kod za frazu koja se upravo formira. Dekompresor tada može da izračuna frazu iz prethodnog stanja.

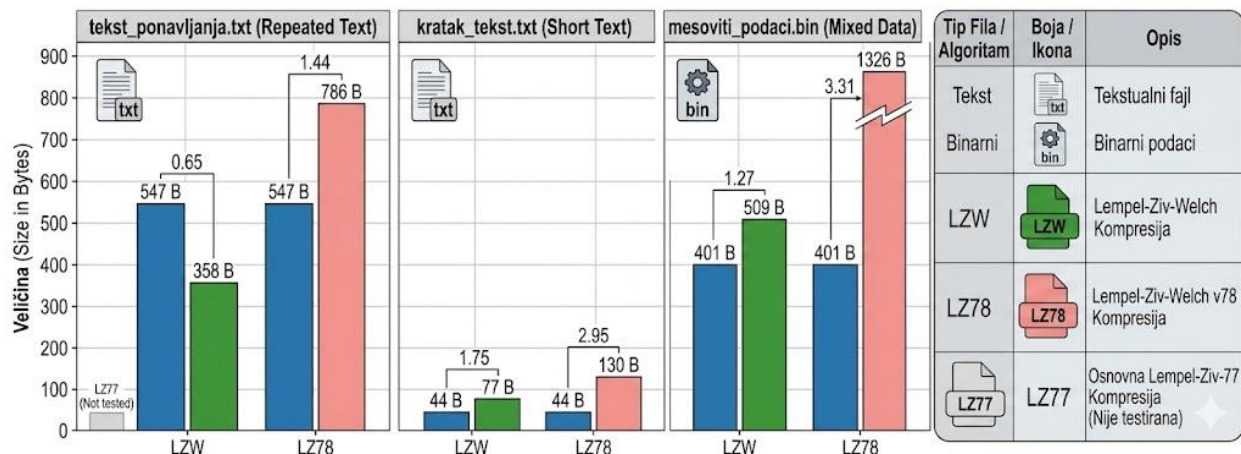
11. Rezultati izvršenja programa

Za testiranje su pripremljena tri ulazna fajla:

- tekst_ponavljanja.txt - tekst sa često ponovljenim frazama
- kratak_tekst.txt - kratak tekstualni fajl
- mesoviti_podaci.bin - binarni fajl sa mešovitim vrednostima

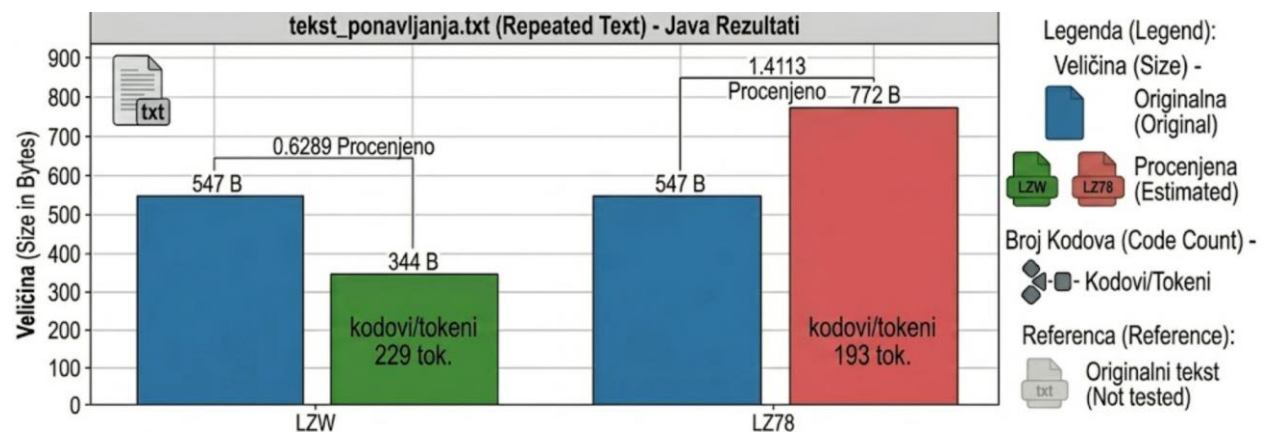
Očekivanje je da algoritmi najbolje rade nad tekstom koji sadrži ponavljanja. Kratki fajlovi i binarni podaci sa malo pravilnosti mogu dati lošiji odnos kompresije.

Rezultati Python analize:



Slika 11: Grafički prikaz rezultata(Python)

Rezultati Java analize:



Slika 12: Grafički prikaz rezultata(Java)

Napomena: Java program čuva kodove u tekstualnom fajlu radi čitljivosti, pa stvarna velicina tog fajla nije isto što i procenjena kompresovana veličina. Za poređenje je uzeta procena koja odgovara načinu pakovanja kodova. Kod LZW algoritma uzeto je 12 bita po kodu.

Dodatna provera ispravnosti izvršena je računanjem hash vrednosti. Originalni fajl `tekst_ponavljanja.txt` i oba Java dekomprimovana izlaza imaju isti hash:

```
ABA94D2760D312EAF7EA5EBCB4DD5F114FE39A76CD20A4F5F9AEEDA152AD1AE6
```

To pokazuje da je dekompresija vratila isti sadržaj koji je bio na ulazu.

12. Poređenje LZ78 i LZW algoritma

LZ78 i LZW pripadaju istoj porodici algoritama i koriste dinamički rečnik, ali se razlikuju po strukturi izlaza.

Kod LZ78 algoritma izlazni token sadrži indeks prefiksa i sledeći simbol. To je vrlo jasno za objašnjenje, jer se direktno vidi kako se nova fraza dobija iz stare fraze i jednog dodatnog simbola. Međutim, cena takvog zapisa je veći broj podataka u izlazu. U našoj implementaciji svaki token zauzima četiri bajta, što je nepovoljno kod malih fajlova.

Kod LZW algoritma izlaz sadrži samo kodove iz rečnika. Početni rečnik već sadrži sve pojedinačne simbole, pa nije potrebno uz svaki kod slati i dodatni simbol. Zbog toga LZW u testu sa ponavljanjem daje bolji rezultat od LZ78 algoritma.

Rezultati jasno pokazuju da ulazni podaci imaju veliki uticaj na kompresiju. Kod fajla `tekst_ponavljanja.txt`, LZW smanjuje veličinu sa 547 B na 358 B u Python implementaciji. To je uspešna kompresija. Kod kratkog fajla i mešovityh binarnih podataka rezultat je lošiji, jer nema dovoljno ponavljanja da nadoknade dodatni trošak formata.

LZ78 je zato pogodniji za demonstraciju osnovne ideje rečnika, dok je LZW praktičniji kada se želi bolji zapis nad tekstualnim podacima. Ova razlika je jedan od razloga zašto se LZW koristio u praktičnim formatima, dok se LZ78 češće posmatra kao teorijska osnova.

13. Moguća unapređenja

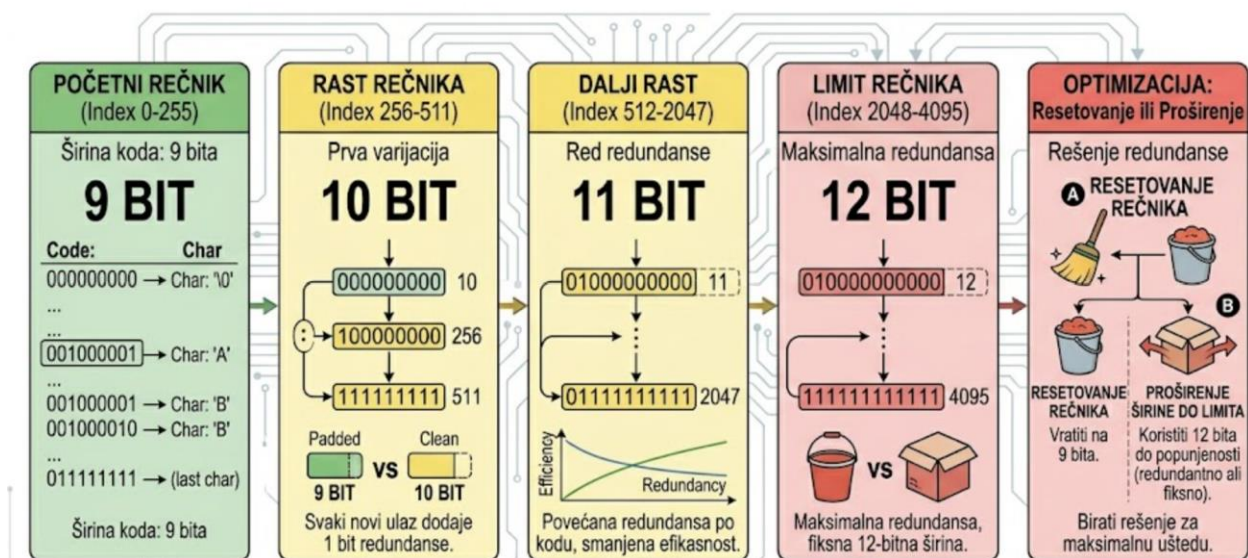
Implementacija u projektu je namerno jednostavna. Cilj je da se algoritmi jasno prikažu, da se vidi rad rečnika i da se omogući poređenje rezultata. Ipak, postoji više mogućih unapređenja.

Prvo unapređenje je promenljiva širina kodova kod LZW algoritma. Umesto da se svi kodovi čuvaju kao 12-bitne vrednosti, moguće je krenuti od 9 bitova i povećavati širinu kako rečnik raste. Na primer, kodovi od 0 do 511 mogu se zapisivati sa 9 bitova, zatim kodovi do 1023 sa 10 bitova, do 2047 sa 11 bitova i do 4095 sa 12 bitova. Ovakav pristup smanjuje broj potrošenih bitova na početku kompresije.

Drugo unapređenje je resetovanje rečnika. Ako se rečnik popuni, algoritam može nastaviti sa statičnim rečnikom, ali može i pratiti odnos kompresije. Ako se odnos pogorša, rečnik se može obrisati i ponovo inicijalizovati. Na taj način algoritam ostaje prilagodljiv kada fajl ima delove sa različitom strukturom.

Treće unapređenje je efikasnije pakovanje LZ78 tokena. U našoj implementaciji tokeni su zapisani jednostavno, kao parovi od po 16 bitova. U realnijem formatu širina indeksa mogla bi da zavisi od veličine rečnika, a marker kraja mogao bi se rešiti drugačije.

Četvrto unapređenje je upoređivanje sa drugim metodama kompresije, na primer sa Huffmanovim kodiranjem ili ZIP arhiviranjem. Time bi se jasnije videlo gde su rečničke metode jake, a gde imaju ograničenja.



Slika 13: Promenljiva širina LZW kodova

14. Zaključak

Kompresija podataka je važan postupak u sistemima za obradu i prenos informacija. Kod kompresije bez gubitaka cilj je da se smanji veličina zapisa, ali da se posle dekompresije dobije potpuno isti sadržaj. Rečničke metode postižu taj cilj tako što pronalaze ponavljajuće obrasce i zamenjuju ih kraćim referencama.

U radu su analizirani i implementirani LZ78 i LZW algoritam. LZ78 pokazuje osnovnu ideju dinamičkog rečnika kroz tokene oblika indeks-prefiksa i sledeći simbol. LZW koristi početni rečnik pojedinačnih simbola i na izlaz šalje samo kodove, zbog čega u praksi često postiže bolji odnos kompresije.

Praktični deo projekta potvrđuje teorijska očekivanja. Tekst sa mnogo ponavljanja dobro se kompresuje LZW algoritmom. Kratki fajlovi i mešoviti binarni podaci daju slabije rezultate, jer dodatni trošak zaglavlja i kodova može biti veći od dobitka. To pokazuje da uspeh kompresije zavisi od strukture ulaznih podataka.

Implementacije u Pythonu i Javi omogućavaju da se isti principi sagledaju iz dva ugla. Python implementacija pravi binarni projektni format i CSV analizu, dok Java implementacija daje jednostavan, čitljiv prikaz kodova i dekomprimovanog izlaza.

15. Literatura

1. Abraham Lempel, Jacob Ziv, "Compression of Individual Sequences via Variable-Rate Coding", IEEE Transactions on Information Theory, 1978.
2. Terry A. Welch, "A Technique for High-Performance Data Compression", Computer, 1984.
3. David Salomon, Data Compression: The Complete Reference, Springer.
4. Guy E. Blelloch, Introduction to Data Compression, Carnegie Mellon University.
5. Materijali sa predavanja iz predmeta Metodi i sistemi za obradu signala.